

Integration of Classy-Blocks and OpenFOAM within Blender

B C H Benjamin

Weekly Progress Report

OpenFOAM GUI

FOSSEE, IIT Bombay

April 26, 2026

- 1 Introduction
- 2 Research Gap
- 3 Problem Statement
- 4 Geometry Export & Pipeline Routing
- 5 Advanced Block Projection
- 6 Universal Shape Support (Week 7)

Introduction

- ◀ **OpenFOAM:** A C++ toolbox for CFD using finite volume methods.
- ◀ **Classy-Blocks:** A Python library for procedural `blockMeshDict` generation, mimicking modern 3D CAD modeling.
- ◀ **Blender Integration:** Utilizing Blender as a frontend for geometry extraction and mesh visualization using `pyvista`.

Research Gap

- ◀ **Gap 1:** Lack of native Blender 5.x support for advanced Python meshing libraries like `classy_blocks`.
- ◀ **Gap 2:** Absence of an integrated “One-Click” solution for geometry extraction, mesh building, and quality reporting within a unified environment.

Problem Statement

- ◀ To develop a Blender Add-on that leverages `classy_blocks` and `pyvista` to automate OpenFOAM case setup, allowing users to extract geometry, generate high-quality meshes, and execute solvers without leaving the Blender interface.

Work Done: STL Extraction & Dynamic Routing

- ◀ **Terrain Export Operator:** Developed `MESH_OT_export_terrain_stl` to extract Blender's active geometric context (e.g. displaced planes) directly into the active OpenFOAM `constant/triSurface` directory.
- ◀ **Dynamic Spec Routing:** Upgraded `build_from_spec` to automatically dispatch geometry build logic. If a block specifies surface mappings, it seamlessly routes generation to the specialized `build_box_block_with_stl` function.

Work Done: Surface Conformity

- ▶ **STL Face Mapping:** Implemented direct integration with `classy_blocks` `project_side()` API. Users can select any local block face (e.g., top, bottom) and logically bind it to the exported TriSurface STL.
- ▶ **End-to-End Conformity:** Fully wired the feature set to ensure that when `blockMeshDict` is generated, the configured block faces automatically conform to the underlying complex terrain representations within OpenFOAM.
- ▶ **Rigorous Safety Tests:** Developed targeted test suites mocking the block builder safely without requiring mesh instantiation, validating side mappings without crashing out-of-bounds `classy_blocks` constraints.

Week 7: Multi-Shape Block Architecture

- ◀ **Problem:** Previous system rendered ALL shapes as rectangular boxes — cylinders, spheres, and planar circles either degraded badly or disappeared after reload.
- ◀ **Solution:** Implemented a **Two-Layer Topological Auto-Detection Pipeline**:
 - 1 **Layer 1 (Heuristics):** Compare mesh volume to bounding box volume to classify shapes (Box ≈ 1.0 , Cylinder ≈ 0.785 , Sphere ≈ 0.523).
 - 2 **Layer 2 (PyVista Validation):** Export temp STL and validate geometry (manifoldness, unique normals, curvature, isotropic sphere bounds).
 - 3 Use classy_blocks native primitives (cb.Box, cb.Cylinder, paired cb.Hemisphere) instead of blind STL projection.
 - 4 Planar circles are converted into thin structured disks; unsupported meshes now trigger a small non-blocking warning and are skipped safely.
- ◀ **Result:** Resolved the critical OpenFOAM crash (“*Cannot find triSurface file...*”). Standard primitives now generate natively without STL mapping.

Week 7: Block Type Dispatch

- ◀ **Advanced users** can also select block type explicitly:
 - BOX → `cb.Box(p_min, p_max)` + optional self-projection
 - EXTRUDE → `cb.Extrude(cb.Face(4pts), vector)`
 - REVOLVE → `cb.Revolve(cb.Face(4pts), angle, axis, origin)`
- ◀ **Key API Discovery:** `cb.Extrude` and `cb.Revolve` require `cb.Face()` wrapper — direct list arguments cause `AttributeError`.

Week 7: Critical Bug Fixes

- ◀ **Bug 1 — bpy_prop_array `TypeError`:** `obj.bound_box` returns `bpy_prop_array`, not `Vector`. Matrix multiplication crashed for every object. **Fix:** Wrap in `mathutils.Vector()`.
- ◀ **Bug 2 — Geometry Section Missing:** `mesh.add_geometry()` was not called before `project_side()`. `blockMesh` crashed with *"Cannot find surface in geometry"*. **Fix:** `_register_geometry()` with correct format.
- ◀ **Bug 3 — Case Path:** Trailing slashes + non-existent default path. **Fix:** `rstrip('/') + os.makedirs(exist_ok=True)`.
- ◀ **Bug 4 — Cylinder Reload Drift:** generic chopping on round shapes distorted cylinder output after regeneration. **Fix:** use round-specific chop helpers for cylinders and hemispheres.
- ◀ **Bug 5 — Silent Shape Loss:** spheres and circles could vanish with no user feedback. **Fix:** sphere meshing added; planar circles become thin disks; unsupported meshes now raise a UI warning.

Week 7: Files Modified & Test Results

- ◀ **geometry_extractor.py:** Added sphere support, planar-circle-to-disk support, and non-blocking unsupported warnings on top of the 2-layer detector.
- ◀ **operators.py:** Removed obsolete primitive STL export logic and surfaced unsupported-mesh warnings in the operator flow.
- ◀ **mesh_builder.py:** Added `_build_sphere()`, `_build_disk()`, and round-specific chopping for stable cylinder/sphere output.
- ◀ **properties.py / ui.py:** Added a stored warning banner so unsupported meshes are visible in the panel without blocking the run.
- ◀ **Tests:** Added extractor and builder regressions for sphere, disk, and unsupported-skip handling.
- ◀ **Test Results:** 49 automated tests collected; focused Week 7 suites passing.